TRANSFORMED 2 SUCCEED

transformed2succeed.com

FREE LIVE SESSION · 30 MINUTES

FinOps for AI-ML Systems

Read an AI cloud bill like an owner, cut the waste without hurting the model, and keep the savings.

PRESENTER Emmanuel N. **LIVE** Mondays & Thursdays · 7:30 PM Central · 30 min

TONIGHT'S 30 MINUTES

Welcome! Here's the plan

0–2 min

Welcome

Say hi in the chat: your name, where you're joining from, and your role.

2–20 min

Three modules

One real idea from each module of FinOps for AI-ML Systems, with the lab behind it.

20–23 min

Keep going

How to get the full course, every lab, and support along the way.

23–30 min

Your questions

Live Q&A. Drop questions in the chat any time; I'll answer at the end.

WHY THIS COURSE MATTERS

AI bills leak, a little every hour, from things nobody owns

01

GPUs bill by the hour

Busy or idle, the meter runs. A forgotten notebook GPU costs as much as a busy one.

02

LLMs bill by the token

Every word in and out has a price. Usage grows quietly with success.

03

Experiments sprawl

Notebooks, snapshots, test endpoints: created fast, deleted never, tagged rarely.

THE COURSE MAP

Inform, Optimize, Operate: three modules

MODULE 1

See Every Dollar

Read a month of AI bills: where the money goes, who owns it, what one prediction costs.

MODULE 2

Cut the Waste, Not the Model

Stop idle GPUs, train on spot safely, and cut the LLM bill above a quality floor.

MODULE 3

Budgets That Bite

Forecast every team's month, catch spikes on day one, and gate new spend before it exists.

01

MODULE 1 OF 3

See Every Dollar

Read an AI cloud bill like an owner. You can't cut what you can't see.

Where this month's $5,778 went

$2,160

gpu.small: the notebook GPUs, 37% of the bill

$1,384

gpu.large: actually training the model

$1,214

llm.large: the AI assistant, paid per token

28%

of all spend has no owner tag at all

The notebooks cost more than training the model. Nobody decided that. Nobody was looking.

Three tags, on everything, from day one

01

team

Who pays and who gets the alert: fraud, research, support.

02

model

Which AI system it serves: fraud-v2, assistant. Needed for unit costs.

03

env

dev or prod. Different rules: dev GPUs must stop themselves.

Target: at most 5% of spend untagged. Our lab starts at 28%. No tag = no bill, no alert, nobody turns it off.

Your mission: read the bill like an owner

1

Where does the money go?

top.py: the total, cost by SKU, and the top 5 resources.

2

Who owns this?

tag_audit.py: untagged share. PASS at 5% or less, else exit 1.

3

Fix tags, then showback

Apply tag fixes, then show each team its own bill. UNALLOCATED stays visible.

4

What does one unit cost?

unit_cost.py: cost per 1,000 predictions and per chat.

Give every dollar an owner

TERMINAL

```
$ python3 lab/make_data.py
$ python3 module-1/tag_audit.py; echo "exit $?"
$ cp module-1/tags.example.json module-1/tags.json
$ python3 module-1/showback.py
```

`make_data.py`
a month of pretend bills; same seed, same numbers

`tag_audit.py`
28% untagged: RESULT FAIL, exit 1

`tags.json`
fills in missing tags only

`showback.py`
each team's bill, UNALLOCATED kept visible

YOU SHOULD SEE RESULT: PASS after the fix, and research as the biggest spender at $2,273, more than the production fraud team.

Three numbers your CFO will understand

$0.077

per 1,000 fraud predictions, to serve

$0.270

per 1,000 predictions, including training

$0.012

per assistant chat

Total cost rises with success. Unit cost shows whether you're getting more efficient, or just bigger.



You can't cut what you can't see.

Where, who, and per what. Every month. Then ask of every new resource: who will turn this off?

02

MODULE 2 OF 3

Cut the Waste, Not the Model

Idle GPUs, spot training, cheaper LLM calls: with a quality floor.

Waste has three shapes: look in this order

01

Idle

Paying for capacity that does nothing: notebooks overnight, forgotten GPUs. Easiest win.

02

Overpriced

Paying full price for work that could run on cheaper, interruptible capacity.

03

Oversized

A huge model, or a huge machine, for a small job.

In the lab, idle.py finds ≈ $1,897 a month: two notebooks to AUTO-STOP, one to DELETE.

Spot is cheap only if you can survive losing it

ON-DEMAND

$4.00 / hour

Yours until you stop it. Full price. Right for production endpoints. In the sim: 40 h, $160.

VS

SPOT

$1.20 / hour

Spare capacity that can be taken back. No checkpoints: $275, 24% never finish. Hourly checkpoints: $52.

Lab prices, 500 simulated runs each. Same discount, opposite results: spot savings come from your code, not the price list.

Cheaper LLM bills, same quality floor

$1,214

all large model: 97.9% good answers · PASS

$121

all small model: 90.2% good answers · FAIL

$290

route easy questions to small: 97.2% · PASS

$102

route + cache good answers: 98.9% · PASS

Floor: at least 95% good answers. Route + cache is 92% cheaper with better quality. Never cache anything personal.



“What's the worst quality we'll accept?”

Agree on it before you look at the savings. A saving that breaks the floor isn't a saving: it's a cost moved onto your customers.

Your mission: find $1,897 and cut the LLM bill

1

Find the idle GPUs

idle.py: busy vs idle hours per GPU, DELETE / AUTO-STOP / ok, and the saving.

2

Is spot worth it?

spot_sim.py: on-demand vs spot vs spot + checkpoints, 500 runs each.

3

Cut the LLM bill

llm_router.py: four strategies on 2,000 requests, against a 95% floor.

Pay for work, not for waiting

TERMINAL

```
$ python3 module-2/idle.py
$ python3 module-2/spot_sim.py
$ python3 module-2/llm_router.py
```

idle.py
an hour is busy at 10%+ GPU use

spot_sim.py
lost work + restart time, billed on the clock

llm_router.py
verdict = PASS only if quality >= 0.95

YOU SHOULD SEE notebook-gpu-03 DELETE · Estimated saving: $1,897 per month · route + cache $102 98.9% PASS

03

MODULE 3 OF 3

Budgets That Bite

Forecasts, alerts and a cost gate that keep the savings.

A clean-up vs a system

A CLEAN-UP

Once a quarter

Someone finds the waste, deletes it, sends a happy email. New waste starts the next day. Savings fade.

VS

A SYSTEM

Every day, automatically

Forecasts, alerts to owners, and a gate on new resources. Waste is caught as it starts. Savings stick.

Day 20 of 30: who will blow the budget?

374%

research: forecast $2,988 vs an $800 budget · OVER

96%

support: the assistant keeps growing · WARN

86%

fraud: on track · OK

36x

research spike on day 74: $791 vs a normal $22

Forecast = spent so far + median of the last 7 days × days left. Spike = over 1.5× normal AND at least $50 more.

Three rules, checked before anything is created

01 **Every resource tagged**
team, model, env, or FAIL. Module 1, enforced.

02 **Dev GPU → auto_stop**
No always-on dev GPUs. Module 2, enforced.

03 **Fits the budget**
Forecast + new cost ≤ budget, else owner approval. Not 'no': 'ask first'.

04 **change-ok.json**
A 4th fraud-api server: +$144 a month. Tagged, fits. PASS, exit 0.

05 **change-bad.json**
4 big always-on GPU notebooks: +$11,520 a month. FAIL ×3, exit 1.

The cheapest dollar is the one you never start spending.

Forecast, alert, gate

TERMINAL

```
$ python3 module-3/forecast.py
$ python3 module-3/anomaly.py
$ python3 module-3/cost_gate.py module-3/change-bad.json
$ python3 -m unittest discover tests
```

forecast.py
OK / WARN / OVER / NO OWNER per team

anomaly.py
spike → alert to the owner in budgets.json

cost_gate.py
tags, auto_stop, budget headroom; exit 0/1

unittest
6 robot checks on the guardrails

YOU SHOULD SEE ALERT day 74: research spent $791 (normal $22, 36x) · change-bad: +$11,520 per month, FAIL ×3 · Ran 6 tests OK

Daily, weekly, monthly, on every change

1

Daily: anomalies

anomaly.py → alerts to owners. Fixed the same day.

2

Weekly: showback

Each team sees its bill and unit cost. 15 minutes.

3

Monthly: forecast + budget

WARN/OVER teams decide: cut, or raise the budget with a reason.

4

Every change: the gate

cost_gate.py in CI. Tags and budgets checked before anything exists.



Every dollar needs an owner and an alarm.

Forecast it, alert on it, and gate it, so the savings stay saved.

RECAP

The whole course on one slide

You can't cut what you can't see

Module 1: where, who, per what. Three tags, showback, unit cost.

Pay for work, not for waiting

Module 2: idle GPUs, spot with checkpoints, route and reuse, quality floor.

Every dollar: an owner and an alarm

Module 3: forecast, alert, gate, and a weekly rhythm.



Find its owner, its unit cost, and its alarm.

Pick one system you touch this week. Whatever's missing is your first FinOps win, and a great story for your next interview.

THE MEMBERSHIP

Every course. One price.

$49

per month

All 8 courses, every module and every lab, plus a live session with me every month.

ALL 8 COURSES INCLUDED

- Linux for AI-ML Systems
- Python for AI-ML
- Zero to AI-ML Systems Engineering
- DevOps for AI-ML Systems
- SRE for AI-ML Systems
- GPU Optimization for AI-ML Systems
- FinOps for AI-ML Systems
- AI Agent Operations

BUILD WITH ME, LIVE

Once a month, we build something together

1

Learn at your pace

Work through any course, any module, on your own schedule.

2

Bring a real problem

An idea, a project at work, or a lab you want to take further.

3

Meet me live

Once a month we get on a call and build it together, step by step.

4

Leave with it built

Something real you made, understand and can show.

FREE LIVE SESSIONS

Every Monday and Thursday, 7:30 PM Central

A different course each session. The full rotation takes 4 weeks, then starts again.

	WEEK 1	WEEK 2	WEEK 3	WEEK 4
MONDAY	Linux for AI-ML Systems	Zero to AI-ML Systems Engineering	SRE for AI-ML Systems	TONIGHT FinOps for AI-ML Systems
THURSDAY	Python for AI-ML	DevOps for AI-ML Systems	GPU Optimization for AI-ML Systems	NEXT AI Agent Operations

Join the membership today

$49/month · all 8 courses · a live build session with me every month

transformed2succeed.com

Next free session: Thursday, 7:30 PM Central · AI Agent Operations

Questions? Let's take them now.

Emmanuel N. · Transformed 2 Succeed